

Assignment : 1

Q. Write a program to add two numbers

```
echo "Enter the first number:"  
read a  
echo "Enter the second number:"  
read b  
sum=$((a+b))  
echo "Sum=$sum"
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~  
$ bash add.sh  
Enter the first number:  
10  
Enter the second number:  
20  
Sum=30
```

Assignment : 2

Q. Write a program to check whether a number is even or odd

```
echo "Enter a number:"  
read num  
if [  $\$(num\%2)$  -eq 0 ]  
then  
    echo "$num is even."  
else  
    echo "$num is odd."  
fi
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~  
$ bash even_odd.sh  
even_odd.sh: line 1: i: command not found  
Enter a number:  
4  
4 is even.  
even_odd.sh: line 10: i: command not found
```

Assignment : 3

Q. Write a program to find highest of three numbers

```
echo "Enter three numbers:"
read a b c
if [ "$a" -ge "$b" ] && [ "$a" -ge "$c" ]; then
    echo "$a is the highest."
elif [ "$b" -ge "$a" ] && [ "$b" -ge "$c" ]; then
    echo "$b is the highest."
else
    echo "$c is the highest."
fi
```

Output :

```
smiiti@DESKTOP-SMITI MINGW64 ~
$ bash highest.sh
Enter three numbers:
10 45 34
45 is the highest.
```

Assignment : 4

Q. Write a program to implement Fibonacci series.

```
echo "Enter the number of terms:"
```

```
read n
```

```
a=0
```

```
b=1
```

```
echo "Fibonacci Series:"
```

```
for ((i=1; i<=n; i++))
```

```
do
```

```
    echo -n "$a "
```

```
    temp=$((a + b))
```

```
    a=$b
```

```
    b=$temp
```

```
done
```

```
echo
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~
```

```
$ bash fibonacci.sh
```

```
Enter the number of terms:
```

```
10
```

```
Fibonacci Series:
```

```
0 1 1 2 3 5 8 13 21 34
```

Assignment : 5

Q. Write a program to find whether a numbers is palindrome or not.

```
echo "Enter a number:"
read num
temp=$num
rev=0
while [ $temp -gt 0 ]
do
    rem=$((temp % 10))
    rev=$((rev * 10 + rem))
    temp=$((temp / 10))
done
if [ $num -eq $rev ]
then
    echo "$num is a palindrome."
else
    echo "$num is not a palindrome."
fi
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~
$ bash palindrome.sh
Enter a number:
101
101 is a palindrome.
```

Assignment : 6

Q. Write a program to compare two strings both the strings are same or not.

```
echo "Enter first string:"
read str1
echo "Enter second string:"
read str2
if [ "$str1" = "$str2" ]
then
    echo "Both strings are equal."
else
    echo "Strings are not equal."
fi
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~
$ bash comparison.sh
comparison.sh: line 1: i: command not found
Enter first string:
madam
Enter second string:
madam
Both strings are equal.
```

Assignment : 7

Q. Write a program to check whether a string is palindrome or not.

```
echo "Enter a string:"
read str
rev=""
len=${#str}
for ((i=len-1; i>=0; i--))
do
    rev="$rev${str:i:1}"
done
if [ "$str" = "$rev" ]
then
    echo "\"$str\" is a palindrome."
else
    echo "\"$str\" is not a palindrome."
fi
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~
$ bash string_palindrome.sh
Enter a string:
eye
"eye" is a palindrome.
```

Assignment : 8

Q. Write a program to check what type of file is it.

```
echo "Enter a file name:"  
read x  
if [ ! -e "$x" ]  
then  
    echo "File does not exist."  
    exit  
fi  
if [ -f "$x" ]  
then  
    echo "This is an ordinary file."  
fi  
if [ -d "$x" ]  
then  
    echo "This is a directory."  
fi
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~  
$ bash filecheck.sh  
Enter a file name:  
test.txt  
This is an ordinary file.  
This is a readable file.  
This is a writable file.
```

Assignment : 9

Q. Write a program to print sub directory under current directory.

```
echo "Sub-directories in the current directory are:"
```

```
for dir in */
```

```
do
```

```
    echo "$dir"
```

```
done
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~
```

```
$ bash subdir.sh
```

```
Sub-directories in the current directory are:
```

```
AppData/
```

```
Application Data/
```

```
Contacts/
```

```
Cookies/
```

```
Documents/
```

```
Downloads/
```

```
Favorites/
```

```
IntelGraphicsProfiles/
```

```
Jedi/
```

```
Links/
```

```
Local Settings/
```

```
My Documents/
```

```
NetHood/
```

```
OneDrive/
```

```
Oracle/
```

```
Postman/
```

```
PrintHood/
```

```
PyCharmMiscProject/
```

Assignment : 10

Q. Write a program to display the file content.

```
echo "Enter the file name:"  
read file  
if [ -f "$file" ]  
then  
    echo "Contents of the file are:"  
    cat "$file"  
else  
    echo "File does not exist."  
fi
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~  
$ bash content.sh  
Enter the file name:  
display.txt  
Contents of the file are:  
OS practical
```

Assignment : 11

Q. Write a program to perform arithmetic operation using switch case.

```
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
echo "Enter your choice:"
read i
echo "Enter two numbers:"
read x y
case $i in
1)
    z=`expr $x + $y`
    echo "Sum = $z"
    ;;
2)
    z=`expr $x - $y`
    echo "Difference = $z"
    ;;
3)
    z=`expr $x \* $y`
    echo "Product = $z"
    ;;
4)
    z=`expr $x / $y`
    echo "Division = $z"
```

```
;;  
*)  
    echo "Invalid Choice"  
;;  
Esac
```

Output :

smi@DESKTOP-SMITI MINGW64 ~

\$ bash switch.sh

1. Addition

2. Subtraction

3. Multiplication

4. Division

Enter your choice:

3

Enter two numbers:

5 6

Product = 30

Assignment : 12

Q. Write a program to print process id of a process and its parent process id also.

```
echo "Process ID (PID) = $$"
```

```
echo "Parent Process ID (PPID) = $PPID"
```

Output :

```
smiti@DESKTOP-SMITI MINGW64 ~
```

```
$ bash process.sh
```

```
Process ID (PID) = 808
```

```
Parent Process ID (PPID) = 782
```

Assignment : 13

Q. Write a program to fork a new child process to display list of files and parent process should wait for the completion of child.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    /* fork another process */
    pid = fork();

    if (pid < 0)
    {
        /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0)
    {
        /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
}
```

```
else
{
    /* parent process */
    /* parent will wait for the child */
    wait(NULL);
    printf("Child Complete\n");
}

return 0;
}
```

Output :

```
smiiti@DESKTOP-SMITI:~$ gcc fork.c -o fork
```

```
smiiti@DESKTOP-SMITI:~$ ./fork
```

```
fork fork.c
```

```
Child Complete
```

Assignment : 14

Q. Write a program to create a new process using system() that displays the processes running on your system.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    printf("Processes running on the system:\n");
    system("ps");
    return 0;
}
```

Output :

```
smiti@DESKTOP-SMITI:~$ nano process.c
smiti@DESKTOP-SMITI:~$ gcc process.c -o process
smiti@DESKTOP-SMITI:~$ ./process
```

Processes running on the system:

PID	TTY	TIME	CMD
2793	pts/3	00:00:00	bash
2936	pts/3	00:00:00	process
2937	pts/3	00:00:00	sh
2938	pts/3	00:00:00	ps

Assignment : 15

Q. Write a program to duplicate a Program's Process using fork().

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    printf("This is a process.\n");
    printf("Process ID = %d\n", getpid());
    return 0;
}
```

Output :

```
smiti@DESKTOP-SMITI:~$ nano duplicate.c
smiti@DESKTOP-SMITI:~$ gcc duplicate.c -o duplicate
smiti@DESKTOP-SMITI:~$ ./duplicate
This is a process.
Process ID = 2983
This is a process.
Process ID = 2984
```

Assignment : 16

Q. Write a program to create a Zombie Process.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("Child process\n");
        printf("Child PID = %d\n", getpid());
        exit(0);
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
}
```

```
    sleep(30);  
}  
return 0;  
}
```

Output :

smi@DESKTOP-SMITI:~\$ nano zombie.c

smi@DESKTOP-SMITI:~\$ gcc zombie.c -o zombie

smi@DESKTOP-SMITI:~\$./zombie

Parent process

Child process

Parent PID = 3021

Child PID = 3022

Child PID = 3022

Assignment : 17

Q. Write a c program to simulate the CPU scheduling algorithm First Come First Serve (FCFS)

```
#include <stdio.h>

int main()
{
    int n, i;
    int bt[20], wt[20], tat[20], ct[20];
    float avg_wt = 0, avg_tat = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    // Input burst time
    printf("Enter burst time for each process:\n");
    for (i = 0; i < n; i++)
    {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    // Completion time
    ct[0] = bt[0];
    for (i = 1; i < n; i++)
    {
        ct[i] = ct[i - 1] + bt[i];
    }
    // Waiting time and Turnaround time
    for (i = 0; i < n; i++)
    {
```

```

    tat[i] = ct[i];    // AT = 0
    wt[i] = tat[i] - bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

// Display results
printf("\nProcess\tBurst Time\tCompletion Time\tWaiting Time\tTurnaround Time\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n",
        i + 1, bt[i], ct[i], wt[i], tat[i]);
}

avg_wt = avg_wt / n;
avg_tat = avg_tat / n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Output :

Enter the number of processes: 4

Enter burst time for each process:

P1: 5

P2: 3

P3: 8

P4: 6

Process	Burst Time	Completion Time	Waiting Time	Turnaround Time
---------	------------	-----------------	--------------	-----------------

P1	5	5	0	5
----	---	---	---	---

P2	3	8	5	8
----	---	---	---	---

P3	8	16	8	16
----	---	----	---	----

P4	6	22	16	22
----	---	----	----	----

Average Waiting Time = 7.25

Average Turnaround Time = 12.75

Assignment : 18

Q. Write a program to stimulate the CPU scheduling algorithm Shortest job first (Non- Preemption).

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[20], wt[20], tat[20], ct[20], p[20];
    int temp;
    float avg_wt = 0, avg_tat = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    // Input burst time
    printf("Enter burst time for each process:\n");
    for (i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    // Sort processes according to burst time
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (bt[i] > bt[j])
            {
```

```

        // Swap burst time
        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;

        // Swap process number
        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}

// Completion time
ct[0] = bt[0];
for (i = 1; i < n; i++)
{
    ct[i] = ct[i - 1] + bt[i];
}

// Waiting time and Turnaround time
for (i = 0; i < n; i++)
{
    wt[i] = ct[i] - bt[i];
    tat[i] = ct[i];

    avg_wt += wt[i];
    avg_tat += tat[i];
}

```

```
// Display result
printf("\nProcess\tBurst Time\tCompletion Time\tWaiting Time\tTurnaround Time\n");
for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n",
           p[i], bt[i], ct[i], wt[i], tat[i]);
}
avg_wt = avg_wt / n;
avg_tat = avg_tat / n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}
```

Output :

Enter the number of processes: 4

Enter burst time for each process:

P1: 6

P2: 2

P3: 8

P4: 3

Process	Burst Time	Completion Time	Waiting Time	Turnaround Time
P2	2	2	0	2
P4	3	5	2	5
P1	6	11	5	11
P3	8	19	11	19

Average Waiting Time = 4.50

Average Turnaround Time = 9.25

Assignment : 19

Q. Write a c program to simulate the CPU scheduling priority algorithm.

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20], ct[20], p[20];
    int temp;
    float avg_wt = 0, avg_tat = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    // Input burst time and priority
    printf("\nEnter Burst Time and Priority for each process:\n");
    for (i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("\nP%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
```

```

    if (priority[i] > priority[j])
    {
        // Swap priority
        temp = priority[i];
        priority[i] = priority[j];
        priority[j] = temp;

        // Swap burst time
        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;

        // Swap process number
        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}

```

```

// Calculate Completion Time
ct[0] = bt[0];
for (i = 1; i < n; i++)
{
    ct[i] = ct[i - 1] + bt[i];
}

```

```

// Calculate Waiting Time and Turnaround Time
for (i = 0; i < n; i++)
{
    wt[i] = ct[i] - bt[i];
    tat[i] = ct[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

// Display results
printf("\n\nProcess\tBurst Time\tPriority\tCompletion Time\tWaiting Time\tTurnaround Time\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t\t%d\t\t%d\t\t%d\n",
        p[i], bt[i], priority[i], ct[i], wt[i], tat[i]);
}

// Calculate averages
avg_wt = avg_wt / n;
avg_tat = avg_tat / n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Output :

Enter the number of processes: 4

Enter Burst Time and Priority for each process:

P1 Burst Time: 6

P1 Priority: 3

P2 Burst Time: 2

P2 Priority: 1

P3 Burst Time: 8

P3 Priority: 4

P4 Burst Time: 3

P4 Priority: 2

Process	Burst Time	Priority	Completion Time	Waiting Time	Turnaround Time
---------	------------	----------	-----------------	--------------	-----------------

P2	2	1	2	0	2
----	---	---	---	---	---

P4	3	2	5	2	5
----	---	---	---	---	---

P1	6	3	11	5	11
----	---	---	----	---	----

P3	8	4	19	11	19
----	---	---	----	----	----

Average Waiting Time = 4.50

Average Turnaround Time = 9.25

Assignment : 20

Q. Simulate the CPU scheduling algorithm round-robin.

```
#include <stdio.h>

int main()
{
    int n, i, time = 0, tq;
    int bt[20], rem_bt[20], wt[20], tat[20];
    float avg_wt = 0, avg_tat = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time for each process:\n");
    for (i = 0; i < n; i++)
    {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
        wt[i] = 0;
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    // Round Robin Scheduling
    while (1)
    {
        int done = 1;
        for (i = 0; i < n; i++)
        {
            if (rem_bt[i] > 0)
```

```

{
    done = 0;
    if (rem_bt[i] > tq)
    {
        time = time + tq;
        rem_bt[i] = rem_bt[i] - tq;
    }
    else
    {
        time = time + rem_bt[i];
        // Waiting time
        wt[i] = time - bt[i];
        rem_bt[i] = 0;
    }
}

if (done == 1)
    break;
}

// Calculate Turnaround Time
for (i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];
    avg_wt = avg_wt + wt[i];
    avg_tat = avg_tat + tat[i];
}

avg_wt = avg_wt / n;
avg_tat = avg_tat / n;

```

```
// Display results
printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}
```

Output :

Enter the number of processes: 4

Enter burst time for each process:

P1: 5

P2: 4

P3: 2

P4: 7

Enter Time Quantum: 2

Process	Burst Time	Waiting Time	Turnaround Time
P1	5	8	13
P2	4	6	10
P3	2	4	6
P4	7	11	18

Average Waiting Time = 7.25

Average Turnaround Time = 11.75

Assignment : 21

Q. Implement FIFO page replacement technique.

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];
    int n, f, i, j;
    int pointer = 0;
    int page_faults = 0;
    int page_hits = 0;
    int found;
    printf("Enter the number of pages: ");
    scanf("%d", &n);
    printf("Enter the page reference string:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &pages[i]);
    }
    printf("Enter the number of frames: ");
    scanf("%d", &f);
    // Initialize frames with -1
    for (i = 0; i < f; i++)
    {
        frames[i] = -1;
    }
    printf("\nPage\tFrames\t\tStatus\n");
```

```

// FIFO Page Replacement
for (i = 0; i < n; i++)
{
    found = 0;

    // Check whether page is already present
    for (j = 0; j < f; j++)
    {
        if (frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }
    if (found == 0)
    {
        // Page fault
        frames[pointer] = pages[i];
        pointer = (pointer + 1) % f;
        page_faults++;
        printf("%d\t", pages[i]);
        for (j = 0; j < f; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
    }
}

```

```

        printf("\tPage Fault\n");
    }
    else
    {
        // Page hit
        page_hits++;

        printf("%d\t", pages[i]);

        for (j = 0; j < f; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\tPage Hit\n");
    }
}

printf("\nTotal Page Faults = %d\n", page_faults);
printf("Total Page Hits = %d\n", page_hits);

return 0;
}

```

Output :

Enter the number of pages: 12

Enter the page reference string:

1 2 3 4 1 2 5 1 2 3 4 5

Enter the number of frames: 3

Page	Frames	Status
1	1 - -	Page Fault
2	1 2 -	Page Fault
3	1 2 3	Page Fault
4	4 2 3	Page Fault
1	4 1 3	Page Fault
2	4 1 2	Page Fault
5	5 1 2	Page Fault
1	5 1 2	Page Hit
2	5 1 2	Page Hit
3	5 3 2	Page Fault
4	5 3 4	Page Fault
5	5 3 4	Page Hit

Total Page Faults = 9

Total Page Hits = 3

Assignment : 22

Q. Implement LRU page replacement technique.

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];
    int n, f, i, j;
    int page_faults = 0, page_hits = 0;
    int found, pos, oldest;
    printf("Enter the number of pages: ");
    scanf("%d", &n);
    printf("Enter the page reference string:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &pages[i]);
    }
    printf("Enter the number of frames: ");
    scanf("%d", &f);
    // Initialize frames
    for (i = 0; i < f; i++)
    {
        frames[i] = -1;
    }
    printf("\nPage\tFrames\t\tStatus\n");
```

```

// LRU Page Replacement
for (i = 0; i < n; i++)
{
    found = 0;

    // Check whether page is already present
    for (j = 0; j < f; j++)
    {
        if (frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }
    if (found)
    {
        page_hits++;
        printf("%d\t", pages[i]);
        for (j = 0; j < f; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\tPage Hit\n");
    }
}

```

```

else
{
    page_faults++;
    // Check for an empty frame
    pos = -1;

    for (j = 0; j < f; j++)
    {
        if (frames[j] == -1)
        {
            pos = j;
            break;
        }
    }
    // If no empty frame, find LRU page
    if (pos == -1)
    {
        int last_used[10];
        for (j = 0; j < f; j++)
        {
            int k;
            last_used[j] = -1;
            // Find the last occurrence of frames[j]
            for (k = i - 1; k >= 0; k--)
            {
                if (pages[k] == frames[j])
                {
                    last_used[j] = k;
                }
            }
        }
    }
}

```

```

        break;
    }
}
}

// Find the least recently used page
oldest = last_used[0];
pos = 0;
for (j = 1; j < f; j++)
{
    if (last_used[j] < oldest)
    {
        oldest = last_used[j];
        pos = j;
    }
}

// Replace the LRU page
frames[pos] = pages[i];
printf("%d\t", pages[i]);
for (j = 0; j < f; j++)
{
    if (frames[j] == -1)
        printf("- ");
    else
        printf("%d ", frames[j]);
}

```

```

        printf("\tPage Fault\n");
    }
}

printf("\nTotal Page Faults = %d\n", page_faults);
printf("Total Page Hits = %d\n", page_hits);

return 0;
}

```

Output :

Enter the number of pages: 12

Enter the page reference string:

1 2 3 4 1 2 5 1 2 3 4 5

Enter the number of frames: 3

Page	Frames	Status
1	1 - -	Page Fault
2	1 2 -	Page Fault
3	1 2 3	Page Fault
4	4 2 3	Page Fault
1	4 1 3	Page Fault
2	4 1 2	Page Fault
5	5 1 2	Page Fault
1	5 1 2	Page Hit
2	5 1 2	Page Hit
3	5 3 2	Page Fault
4	5 3 4	Page Fault
5	5 3 4	Page Hit

Total Page Faults = 9

Total Page Hits = 3

Assignment : 23

Q. Implement optimum replacement technique.

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];
    int n, f, i, j, k;
    int page_faults = 0, page_hits = 0;
    int found, pos, farthest, future;
    printf("Enter the number of pages: ");
    scanf("%d", &n);
    printf("Enter the page reference string:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &pages[i]);
    }
    printf("Enter the number of frames: ");
    scanf("%d", &f);
    // Initialize frames
    for (i = 0; i < f; i++)
    {
        frames[i] = -1;
    }
    printf("\nPage\tFrames\t\tStatus\n");
```

```

// Optimal Page Replacement
for (i = 0; i < n; i++)
{
    found = 0;

    // Check whether page is already present
    for (j = 0; j < f; j++)
    {
        if (frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }
    if (found)
    {
        page_hits++;
        printf("%d\t", pages[i]);
        for (j = 0; j < f; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\tPage Hit\n");
    }
}

```

```

else
{
    page_faults++;
    // Find an empty frame
    pos = -1;
    for (j = 0; j < f; j++)
    {
        if (frames[j] == -1)
        {
            pos = j;
            break;
        }
    }
    // If no empty frame, find page used farthest in future
    if (pos == -1)
    {
        farthest = -1;
        for (j = 0; j < f; j++)
        {
            future = -1;
            // Search for next use of frames[j]
            for (k = i + 1; k < n; k++)
            {
                if (pages[k] == frames[j])
                {
                    future = k;
                    break;
                }
            }
        }
    }
}

```

```

    }
    // If page is never used again, replace it
    if (future == -1)
    {
        pos = j;
        break;
    }
    // Select the page used farthest in future
    if (future > farthest)
    {
        farthest = future;
        pos = j;
    }
}

// Replace the selected page
frames[pos] = pages[i];
printf("%d\t", pages[i]);
for (j = 0; j < f; j++)
{
    if (frames[j] == -1)
        printf("- ");
    else
        printf("%d ", frames[j]);
}
printf("\tPage Fault\n");
}
}

```

```

printf("\nTotal Page Faults = %d\n", page_faults);
printf("Total Page Hits = %d\n", page_hits);
return 0;
}

```

Output :

Enter the number of pages: 12

Enter the page reference string:

1 2 3 4 1 2 5 1 2 3 4 5

Enter the number of frames: 3

Page	Frames	Status
1	1 - -	Page Fault
2	1 2 -	Page Fault
3	1 2 3	Page Fault
4	1 2 4	Page Fault
1	1 2 4	Page Hit
2	1 2 4	Page Hit
5	5 2 4	Page Fault
1	5 1 4	Page Fault
2	5 1 2	Page Fault
3	3 1 2	Page Fault
4	4 1 2	Page Fault
5	4 5 2	Page Fault

Total Page Faults = 9

Total Page Hits = 3